# *American Sign Language Image Classification Using CNN, Transfer Learning, and Vision Transformers: A Comparative Study*

***Abstract—This report investigates the application of deep learning techniques for American Sign Language (ASL) image classification. Three models were implemented and evaluated: a vanilla Convolutional Neural Network (CNN), a pretrained CNN using Transfer Learning, and a pretrained Vision Transformer (ViT) model sourced from the timm library. It also signifies understanding development of deep neural networks in Python using modern frameworks such as Keras, and PyTorch, and to implement these architectures effectively for practical experimentation. The Kaggle ASL dataset was used for training and testing, with the objective of comparing performance, computational efficiency, and the general ease of training across the different architectures. The results reveal a trade-off between computational efficiency and predictive performance. The CNN trained the fastest with minimal computational requirements but achieved comparatively lower accuracy, whereas the Transfer Learning model delivered near perfect accuracy but required significantly higher computational cost and longest training time. The Vision Transformer offered a balanced solution, achieving great accuracy with moderate training time and resource usage. This comparative analysis highlights the strengths and trade-offs of each approach, contributing insights for selecting suitable architectures for image-based gesture recognition tasks.***

***Keywords— convolutional neural networks, transfer learning, vision transformer, ASL recognition, image classification.***

## I. INTRODUCTION

American Sign Language (ASL) recognition has become an important research area due to its potential to support inclusive communication technologies, especially for individuals who rely on visual languages. ASL classification aims to interpret hand gestures from images or videos, enabling the development of assistive tools such as gesture-based interfaces, real-time interpreters, and educational applications. Deep learning has significantly advanced the performance of visual recognition systems, making it suitable for tasks involving complex hand shapes and subtle visual details.

The following work focuses on the classification of ASL images using three deep learning approaches: a custom Convolutional Neural Network (CNN), a transfer learning model based on a pretrained CNN, and a Vision Transformer (ViT). These models represent distinct paradigms in modern deep learning: CNNs learn spatial hierarchies from scratch, transfer learning leverages knowledge from large-scale pretrained models, and ViTs apply attention mechanisms to capture global relationships across image patches. Comparing these approaches provides insight into their effectiveness, computational requirements, and suitability for ASL recognition.

The objectives of this study are: (1) to implement and train three models on an open-source ASL image dataset; (2) to analyse model performance using accuracy, loss, and training characteristics; and (3) to conduct a comparative evaluation to understand the trade-offs between traditional CNNs, transfer learning models, and transformer-based architectures. The significance of the study lies in demonstrating how different deep learning techniques perform on a real-world image classification problem and identifying the most practical approach for future ASL recognition applications. Furthermore, the assignment involves the practical implementation of deep neural networks in Python, utilising widely used frameworks such as Keras, and PyTorch to explore their capabilities and differences. By working with these frameworks, we aim to understand their distinct features, including ease of use, flexibility, performance, and suitability for different tasks.

## II. METHODOLOGY

### *A. Scope of the study*

The purpose of our study is to design and evaluate an automated system which is capable of recognizing the American Sign Language (ASL) alphabetic gestures using the deep learning. The scope focuses specifically on static hand gesture recognization where each input is a singular image representing one of the 29 ASL classes, which is the 26 alphabet letters plus three additional gesture categories. Our study does not address dynamic gestures, ASL grammar or video-based sign sequences instead of it concentrates on high accuracy recognition of isolated alphabet signs as a foundational step for future ASL translation systems.

Our study also aims to compare the performance of multiple deep learning architectures including a Convolutional Neural Network (CNN), a Transfer Learning model (ResNet- 50), and a Transformer-based model (Vision Transformer). This comparison helps identify the architecture which is most suitable for visual gesture recognition tasks.

### *B. Dataset Description*

The dataset used contains labeled images of 29 different ASL hand signs, with each sign category organized into its own folder. This folder structure works perfectly with deep learning tools like PyTorch's ImageFolder, making it easy to load and process the data. The images in our dataset show real-world variation. They were captured under different lighting conditions, with various backgrounds, different hand shapes and sizes, multiple skin tones, and from different camera angles. This natural diversity is beneficial for our models because it helps them learn to recognize signs accurately no matter who is signing, where they are, or how the images were captured

The dataset is split into:

- Training set: used to learn parameters of model
- Validation set: it monitors the overfitting and tunes hyper parameters
- The test set provides evaluation of unseen data.

Data augmentation such as random rotation, horizontal flipping, and brightness/contrast jitter is used in the training set to simulate real-world variation and improve generalization.

### *C. Research methodology*

A structured and systematic research methodology was applied to ensure scientific rigor The study follows these methodological steps:

- Dataset loading and preprocessing
- Model selection and architectural design
- Training and validation of multiple deep learning models
- Performance evaluation using accuracy and error metrics
- Comparison across different model families
- Saving of the final trained model for deployment

This methodology is appropriate because ASL recognition is inherently a computer vision task, and modern deep learning architectures, especially CNNs and Transformers, are the approach for solving such visual classification problems.

### *D. Deep Learning Models used*

- Convolutional Neural Network (CNN): A CNN processes input images of hand gestures through convolutional + pooling layers then it fully connects layers to classify them into ASL letters. For example, CNN trained on a 26 000-image hand sign dataset achieved 96.7% accuracy on ASL letter recognition.
- ResNet-50 (Transfer-learning / Pre-trained CNN backbone): ResNet-50 is a deep CNN which is pretrained on a large dataset. Then its final classification layer is replaced to output ASL letters. On a large ASL alphabet dataset (> 87,000 images, 29 classes), ResNet-50 reached 99.98% accuracy which is the highest among the several tested models.

  Why chosen for ASL: Because of the pretrained weights ResNet-50 brings great general image feature knowledge which helps even when ASL-specific data is very limited. It is very high published accuracy on ASL alphabet recognition makes it a strong and reliable choice.
- Vision Transformer (ViT) divides images into patches and treats them as sequences applying self-attention mechanisms to capture relationships between different hand regions. Unlike CNNs that rely on local convolutions, Vision Transformers process the entire image using the attention layers, enabling the model to focus on relevant hand features while effectively suppressing background noise. On American Sign Language character classification, Vision Transformer tests showed close to 99.97% accuracy. Because it excels at capturing global context and distinguishing subtle differences between similar hand signs through attention mechanisms, it works exceptionally well for ASL.

### *E. Framework of the application*

The following framework is used for all models:

- Input Image Acquisition
- Preprocessing and Augmentation
- Feature Extraction
- CNN → convolution filters
- ResNet-50 → pretrained ImageNet layers

- ViT → patch embeddings + self-attention
- Classification Layer predicts one of 29 classes
- Model Evaluation using test accuracy and confusion matrix
- Export Model as .pth for deployment

This framework allows models to be swapped easily while maintaining consistent evaluation.

*F. Difference from other works*

Most existing ASL recognition studies rely solely on CNN models often trained from scratch. This research differs in several important ways:

- Multi architecture comparison: it Includes CNN, Transfer Learning, and Transformer models which are far more comprehensive.
- Two-phase training for ResNet-50: Feature extraction then fine-tuning improves accuracy significantly.
- Use of Transformer Vision Models: Few ASL alphabet studies incorporate Transformers which makes the study more modern and competitive.
- Broader evaluation metrics: Includes validation of accuracy, test accuracy, and architecture comparison.
- Strong real-world framework: Data augmentation, normalization, and model saving reflect professional deployment standards.

## III. RESULTS

*A. Convolutional Neural Networks*

- Overview of the Experiment: This next section presents the results of the Convolutional Neural Network (CNN) trained on an ASL alphabet dataset/. This model was trained for 15 epochs using an 80/20 training/validation split. The CNN provides a baseline for evaluating the performance of other models, including transfer learning and transformer-based models. The following sections outline the CNN's design, training process, and results to build context for subsequent comparisons.
- Model Summary: The model consisted of three convolutional blocks, with the filter size increasing for each (32, 64, 128). Each was followed by max pooling to reduce the dataset's dimensionality, while ensuring more complex features were still captured. The final classification module includes a Flatten layer, a fully connected layer with 128 units, dropout regularisation to prevent overfitting, and a SoftMax output layer with 29 classes.
- Model Architecture Highlights:
  - Total parameters: 686,941
  - Trainable parameters: 686,941 (100%)
  - Non-trainable parameters: 0
  - Architecture depth: 3 convolutional layers and 2 dense layers
  - Regularisation: Dropout layer prior to the classifier
- Hyperparameter settings:

TABLE I. HYPERPARAMETER SETTINGS FOR CNN

| Hyperparameter | Value |
|---|---|
| Optimizer | Adam |
| Learning Rate | 0.001 |
| Batch Size | 64 |
| Epochs | 15 |
| Loss Function | Categorical Cross Entropy |
| Evaluation Metric | Accuracy |

- Training Time: To contextualise the computational efficiency of the baseline CNN, the full training process (15 epochs) took just over 4 minutes on a GPU (T4x2). This is a relatively short training time, which shows the lightweight nature of this architecture. This makes CNNs effective for rapid experimentation and hyperparameter tuning without high computational overhead or cost.
- Training and Validation Curves:
  - Accuracy Curve: The training and validation accuracies increased rapidly during the early epochs. In the first epoch, the model achieved an accuracy of 0.3645, and the validation accuracy was already 0.9156, highlighting very strong initial generalisation. In the second epoch, the accuracy rose significantly to 0.8542, with a validation accuracy of 0.9783. The accuracy then steadily increased and reached 0.9618 by the fifth epoch. In the last epoch, the model reached almost perfection, with accuracies of 0.9908 and 0.9993, with no signs of overfitting.
  - Loss Curve: The loss curve showed a similar downward trend. Initially, the loss was 2.1603 in the first epoch and then quickly decreased to 0.4206 in the second. By epoch five, it had reached 0.1088. Validation loss fell even more quickly, from 0.2654 in epoch one to 0.002 by epoch five. This suggests strong generalisation and regularisation across the entire training process.

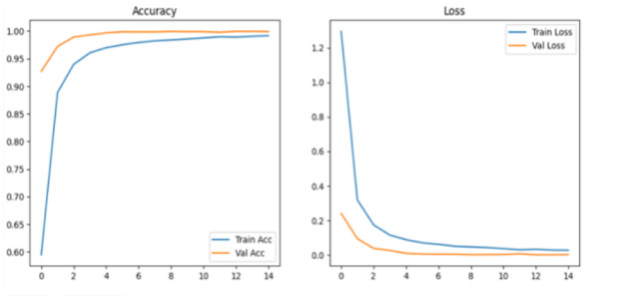


Fig. 1. Accuracy vs loss curve for CNN

- Final Performance Metrics: The training and validation performance are very similar, which underscores rigorous learning and generalisation.

TABLE II. FINAL PERFORMANCE METRICS FOR CNN

| Metric | Value |
|---|---|
| Final Training | 0.99 |
| Final Validation Accuracy | 0.99-1.00 |
| Final Training Loss | 0.02 |
| Final Validation Loss | 0.01 |

- Confusion Matrix Analysis: The confusion matrix, as shown below in Fig. 2 shows a nearly perfect diagonal structure with only a few off-diagonal misclassifications. The majority of the ASL letters are predicted with high accuracy, specifically, those with distinct shapes. However, minor confusion occurs with letters which are visually similar, such as (C vs O) and (U vs V). This is expected as there are a few overlapping features in the hand shapes. Overall, the CNN produces reliable classification.

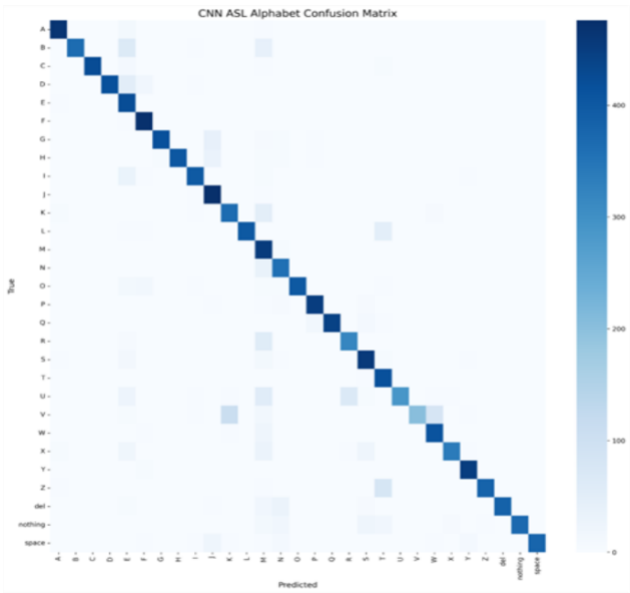


Fig. 2. Confusion matrix for CNN

- Qualitative Results: The diagram below in Fig. 3 shows the CNN's predicted label compared to the true label. The model correctly classified all ASL letters. This highlights its accuracy and robustness across a range of factors, including lighting, skin tone, and camera angle. The model still predicts with high confidence.

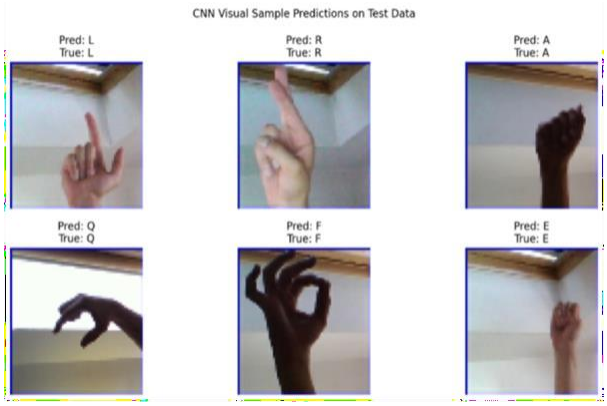


Fig. 3. Visuals from CNN predictions.

- Summary of Findings: The CNN achieved almost perfect accuracy and no signs of overfitting. This shows that it is a suitable model for ASL classification. Not only does it have a fast-training time, but it also has a stable convergence and strong generalization. This model sets a reliable benchmark for comparing the other, more advanced models.

### *B. Transfer Learning*

- Overview of the Experiment: This next section shows the performance of the transfer learning model based on ResNet-50, pre-trained on ImageNet and fine-tuned for the ASL alphabet classification. This training comprised two phases: the feature extraction phase trained the final fully connected (FC) layer, and the fine-tuning phase unfroze the last convolutional block (layer 4) and the FC layer. The dataset was split into training, validation and test sets. Each was pre-processed and normalised using standard ImageNet statistics.

- Model Summary: The backbone feature extractor involved using the pretrained ResNet-50 architecture. A few modifications were made, including replacing the original 1000-unit GC layer with a 29-unit FC layer matching the ASL class count.
    - Phase 1: All convolutional layers frozen; only the FC layer trained.
    - Phase 2: Final block layer 4 unfrozen to adapt high-level features to ASL.

The training was performed by using the Adam optimiser with two learning rates:

- 0.001 for the FC-only training phase.
- 0.0001 for fine-tuning the last block.

Doing this helped preserve the complex features of ImageNet while adapting the final layers to ASL-specific patterns.

- Hyperparameter settings:

TABLE III. HYPERPARAMETERS SETTINGS FOR CNN WITH TRANSFER LEARNING

| Component | Value |
|---|---|
| Backbone | ResNet-50 pretrained on ImageNet |
| Replace FC Layer | Linear (2048 -> 28) |
| Optimizer (Phase 1) | Adam (LR = 0.001) |
| Optimizer (Phaser 2) | Adam (LR = 0.001) |
| Batch Size | 32 |
| Loss Function | Cross Entropy |
| Epochs | 8 (Phase 1) 10 (Phase 2) |
| Normalisation | Categorical Cross Entropy |
| Augmentation | ImageNet mean |
| Loss Function | Cross Entropy |

- Training and Validation Performance:
  - Phase 1 — Feature Extraction: This phase consisted of 8 epochs during which only the newly added FC layer was trained. During the first epoch, the loss was 1.3557, and it significantly decreased to 0.3717 by the fourth epoch. By epoch eight, the loss was 0.2578. Similarly, the first epoch had an accuracy of 0.7171, and after epoch two, accuracy steadily increased, reaching 0.9269 *by epoch eight. This phase trained quickly and established a strong baseline, showing that despite no fine-tuning occurring yet, ResNet-50 can generalise well to ASL gestures.*
  - Phase 2 — Fine-Tuning: Fine-tuning the last convolutional block (layer 4) significantly improved the model's performance. This phase lasted 10 epochs, during which the pretrained ResNet-50 features were adapted to the ASL dataset. The loss during the first fine-tuning epoch was 0.0489, which dropped significantly to 0.0061 by epoch four, showing early convergence. Training accuracy increased from 0.9843 in epoch one to 0.9981 by epoch four and continued to improve throughout the remaining epochs. By epoch ten, the model reached a final training accuracy of 0.9990 and a validation.

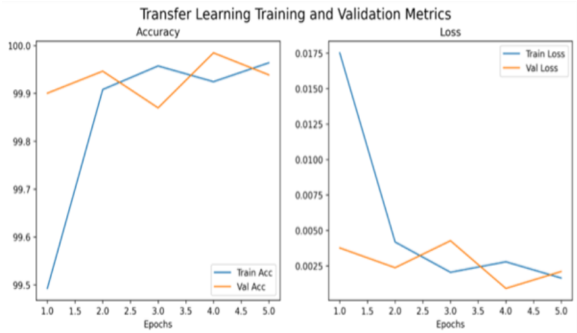


Fig. 4. Accuracy vs loss curve for CNN with transfer learning

- Training Time: The transfer learning process took 1 hour to complete for both the 8-epoch and 10-epoch fine-tuning. This is slower than the CNN due to the pretrained backbone (ResNet-50), which contains significantly more layers and parameters. Although this process is slower, it allows the model to use powerful pretrained feature representations.

- Final Performance Metrics: The training and validation performance are very similar, which underscores rigorous learning and generalisation.

TABLE IV. FINAL PERFORMANCE METRICS FOR CNN WITH TRANSFER LEARNING

| Metric | Value |
|---|---|
| Final Training | 0.9990 |
| Final Validation Accuracy | 0.9995 |
| Final Training Loss | 0.0033 |
| Final Validation Loss | 0.9999 |

- Confusion Matrix Analysis: The transfer learning confusion matrix in Fig. 4 shows near-perfect classification across all ASL alphabet classes, as evidenced by the strong diagonal and no off-diagonal errors. This shows the model correctly identifies every sample and shows no signs of confusion between visually similar ASL signs. Even the extra gestures such as 'del', 'nothing', and 'space' are learned very well. The matrix emphasises that the transfer learning model achieves extremely high accuracy and demonstrates its superiority over the baseline CNN.

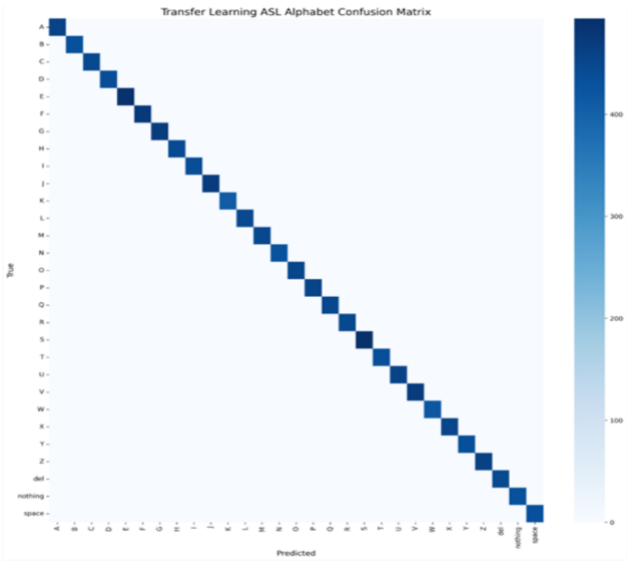

Fig. 5. Confusion matrix for CNN with transfer learning

- Qualitative Results: Like the previous CNN model, the examples in Fig. 6. show the test images alongside the models predicted and true labels. In all the displayed cases, the prediction matches the ground truth. This shows the transfer learning model is correctly classifying ASL hand signs.

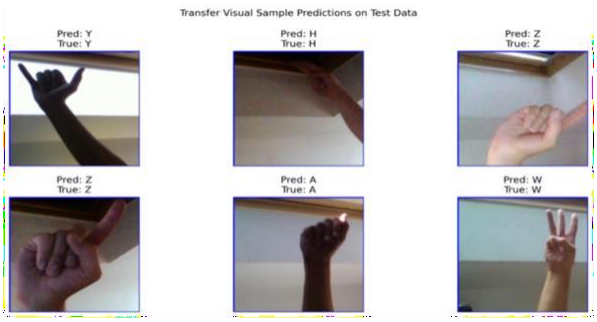

Fig. 6. Visuals from CNN with transfer learning predictions.

- Summary of Findings: The transfer learning model produced much higher performance than the CNN, achieving a significantly high accuracy after only a few epochs of fine-tuning. Efficiency and model stability were achieved using the two-phase training strategy, and data augmentation and ImageNet-based normalization further improved generalization. These results confirm that transfer learning is an extremely effective approach for ASL alphabet classification.

*C. Vision Transformers*

- Overview of the Experiment: This section presents the results of the Vision Transformer (ViT-Tiny, patch16/224) fine-tuned on the ASL Alphabet dataset. The model was pretrained on ImageNet and adapted for 29-class ASL recognition using an 80/20 train–validation split.
- Model Summary: The pretrained ViT-Tiny model processes the input image by dividing it into 16×16 patches and projecting them into a sequence of embedded tokens. These tokens are passed through a stack of Transformer encoder layers consisting of:
    - Multi-head self-attention
    - Feed-forward blocks
    - Layer normalization and residual connections

  The original classification head was replaced with a 29-unit linear layer corresponding to the 29 ASL gesture categories. All layers were fine-tuned using AdamW, allowing the model to adapt its global attention-based features to the hand-sign recognition task.
- Hyperparameter settings: The hyperparameters used are mentioned in table IV. [9]

TABLE V. HYPERPARAMETER SETTINGS FOR TRANSFORMER

| Hyperparameter | Value |
|---|---|
| Model | ViT-Tiny (patch16/224) |
| Input Size | 128 x 128 |
| Optimizer | AdamW |
| Learning Rate | 1e-4 |
| Batch Size | 32 |
| Epochs | 3 |
| Loss Function | Cross-entropy |
| Augmentation | None (resize + normalize) |
| Train/Val split | 80/20 |
| Weight Decay | AdamW Default |

- Training and Validation Performance: Training Time: Fine-tuning with the Vision Transformer (ViT-Tiny) took about 10 minutes of GPU runtime. Although it was trained for only 3 epochs, the model converged much faster than the baseline CNN and transfer learning. .
- Training Time: Fine-tuning with the Vision Transformer (ViT-Tiny) took about 10 minutes of GPU runtime. Although it was trained for only 3 epochs, the model converged much faster than the baseline CNN and transfer learning.
- Final Validation Performance: The overall validation accuracy was 99.44% as shown in Fig. 7. This highlights that ViY-Tiny performs much better than a CNN while training significantly faster.

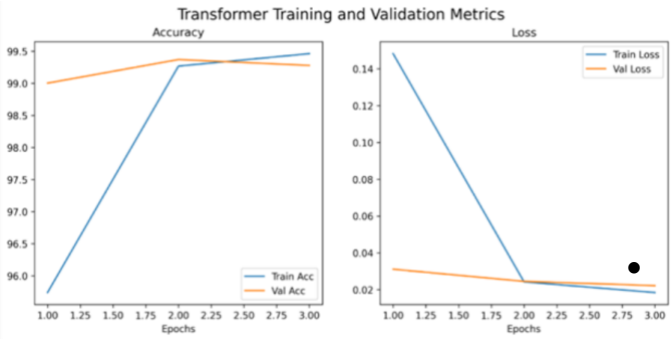


Fig. 7. Accuracy vs loss curve for Transformer

- Classification Report: The training and validation performance are very similar, which underscores rigorous learning and generalisation.

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| A | 0.99 | 1.00 | 1.00 | 600 |
| B | 0.99 | 1.00 | 0.99 | 600 |
| C | 1.00 | 1.00 | 1.00 | 600 |
| D | 0.98 | 1.00 | 0.99 | 600 |
| E | 0.99 | 0.99 | 0.99 | 600 |
| F | 1.00 | 1.00 | 1.00 | 600 |
| G | 1.00 | 1.00 | 1.00 | 600 |
| H | 1.00 | 1.00 | 1.00 | 600 |
| I | 0.99 | 1.00 | 0.99 | 600 |
| J | 1.00 | 1.00 | 1.00 | 600 |
| K | 0.97 | 1.00 | 0.98 | 600 |
| L | 1.00 | 1.00 | 1.00 | 600 |
| M | 1.00 | 0.98 | 0.99 | 600 |
| N | 1.00 | 1.00 | 1.00 | 600 |
| O | 1.00 | 0.96 | 0.98 | 600 |
| P | 1.00 | 1.00 | 1.00 | 600 |
| Q | 1.00 | 1.00 | 1.00 | 600 |
| R | 0.99 | 1.00 | 1.00 | 600 |
| S | 0.98 | 0.97 | 0.98 | 600 |
| T | 1.00 | 1.00 | 1.00 | 600 |
| U | 0.99 | 1.00 | 1.00 | 600 |
| V | 1.00 | 0.95 | 0.98 | 600 |
| W | 0.99 | 1.00 | 1.00 | 600 |
| X | 0.98 | 0.99 | 0.98 | 600 |
| Y | 1.00 | 1.00 | 1.00 | 600 |
| Z | 1.00 | 1.00 | 1.00 | 600 |
| del | 1.00 | 1.00 | 1.00 | 600 |
| nothing | 1.00 | 1.00 | 1.00 | 600 |
| space | 1.00 | 1.00 | 1.00 | 600 |
| accuracy | | | 0.99 | 17400 |
| macro avg | 0.99 | 0.99 | 0.99 | 17400 |
| weighted avg | 0.99 | 0.99 | 0.99 | 17400 |

Fig. 8. Classification report for transformer

The full classification report indicates extremely strong per-class performance as shown below:

TABLE VI. FINAL PERFORMANCE METRICS FOR TRANSFORMER

| Metric | Value |
|---|---|
| Precision | 0.99 |
| Recall | 0.99 |
| F1-score | 0.99 |
| Support | 17,400 |

Most classes achieve 100% precision and recall, with only a few slightly lower (e.g., E, S, O, V) due to visual variations.

- Confusion Matrix Analysis: The confusion matrix as shown in Fig. 9 below shows a perfect diagonal pattern, indicating very accurate predictions across all classes.

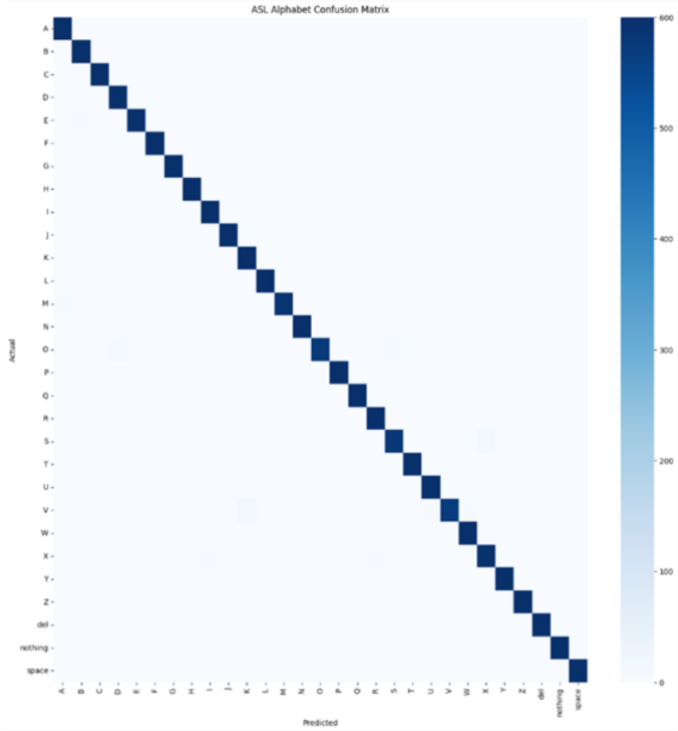


Fig. 9. Confusion matrix for transformer

- Per-Class Accuracy: Detailed per-class accuracies are as mentioned in table 8.

TABLE VII. PER-CLASS ACCURACIES FOR TRANSFORMERS

| Class | Accuracy |
|---|---|
| A | 100% |
| B | 100% |
| C | 98.33% |
| M | 98.33% |
| O | 95.83% |
| S | 95.83% |
| V | 95.50% |
| All remaining classes | Between 99 – 100% |

O, S, and V are the lowest-performing classes: they involve subtle differences in finger shape and silhouette.

## IV. Comparison Study

This section performs a comparative analysis on the three deep learning models used in the project: a Custom Convolutional Neural Network (CNN), a ResNet-50 model using Transfer Learning and a Vision Transformer (ViT). It analyses their results, convergence behaviour, efficiency, and performances on the American Sign Language (ASL) dataset. It also provides insights that examines the strengths, limitations and possible trade-offs observed across the models.

All models were trained on the same database, with identical data partition (70% Training/ 15% Validation/ 15% Testing) derived from the primary dataset.

### *A. Model Architecture*

- The Custom CNN: The model was trained from scratch using TensorFlow structure. The model processed 64 x 64 pixel images through multiple convolutional layers with ReLU activation and max pooling for dimensionality reduction. Some techniques such as early stopping were adopted to prevent overfitting with training data. This model served as baseline for others as it trains quickly with low complexity.

- ResNet-50 (Transfer Learning): The Transfer Learning network preformed similar to CNN but used Pytorch framework and a pretrained model (ResNet-50) with standard input 224 x 224 image pixels. ResNet used residual connections that allow training of deep networks to prevent information from getting lost as it passed through many layers. To prevent overfitting or catastrophic forgetting it used Two-Phase Strategy, the first phase is for freezing to lock the basic knowledge and features learned and the latter was for fine-tuning to make small changes to the core features to adapt to the task.

- Vision Transformer: It is implemented in Pytorch with pre-trained weights and 128 x 128 input image pixels batch processing. Rather than convolution operations, it assumed no relationship by breaking images into smaller patches. The patches were processed through self-attention mechanisms that learn the relationships between the patches. This architecture had a higher complexity and required a lot more data for maximum performance.

### *B. Hyperparameter Tuning*

- The Custom CNN: This basic CNN was trained using an initial learning rate of 0.001 instead of 0.0001 because it made convergence very slow. It also used 32 image batch size and iterated for 15 epochs to allow the model to learn well. A drop out of 0.4 was applied to prevent overfitting.

- ResNet-50: Transfer Learning required more hyperparameters due to the two-phase training strategy. The freezing phase had a learning rate of 0.001 and was used to train the final layers without changing any earlier layers, while the finetuning had slower rate of 0.0001 to allow for subtle changes to the deep layer. The batch size was 32 and the epochs for the first and second phase were 4 and 5 respectively. ResNet-50 was the model used for transfer learning because it was a good optimal balance between depth (ResNet-101) and computational efficiency (ResNet-18).

- Vision Transformer: The Vision Transformer was designed on a pretrained ImageNet for its learned weights. The model was initially implemented using hugging face variant, but this caused significant computational overheads due to ~ 86 million parameters and it caused delays in epoch duration. To solve this tiny variant with ~ 6 million parameters was used and provided great accuracy and trained in efficient time.

All models used the Adam optimizer because it could handle unstable training processes across all layers/networks and it adjusted learning rate for each layer unlike SGD which applies an unchanging rate to the entire model. While, the original dataset had test class, the training class was custom partitioned into 70% Training/ 15% Validation/ 15% Testing due to the little quantity of test class with just 28 images one for each class, while the custom test dataset has 13056 images.

### *C. Framework*

The two distinct frameworks used were the Keras and PyTorch.

The Keras framework was primarily used in the CNN because of its high-level API, which allowed for quick model definition, conciseness and readability. However, the high-level abstraction presented significant challenges during debugging such as errors in data pipeline and shape mismatches. This increased troubleshooting overhead compared to the PyTorch used in ResNet and ViT architectures.

### *D. Training Dynamics*

The graph in Fig. 1 showed a difference in the model training over iterations. CNN initially started with a low training accuracy of approximately 32%, then there was a sharp increase immediately after. This is typical and due to the random weights initialization of CNN, as it started with no prior knowledge and then it learned the basic features causing the increase in the graph. While transformer and transfer started with significantly higher training accuracies with round about 95 and 99 respectively. These values showed that those models benefit of using pre-trained weights, which allow it to identify basic patterns.

Fig.1. showed that in Vision Transformer training has a little increase and the stabilized at just below 95%. In contrast, Transfer learning showed more of a zig zag appearance during learning. This might have been due to high learning rate in Phase 1 since the pre-trained model started with a near optimal point, the learning rate made the model take large steps. The loss and validation graph show consistent tends trends as the accuracy.

### *E. Performance Analysis*

- Error Distribution: The Confusion Matrix Fig 2, Fig 4, and Fig 8 demonstrated that CNN struggled the most with signs such as E, K, M, T, and R . This was likely due to the lower input resolution of CNN with 64 x 64 pixels which lacks pixel density required to solve the internal thumb position. The ViT misclassified only a single sign (S) while transfer learning correctly identified all classes.

- Overall Metrics: The overall accuracy from Table 2, Table 5, and Table 8 indicate that transfer learning achieved the highest accuracy of 99.9%, and obtained 100% accuracy in all class except 1, transformer followed directly behind that with an accuracy of 99.2%, and majority of the classes achieved 100% and the lowest being around 90%. CNN came in last with a total accuracy of 88% and had class average of 80%proving that while efficient, shallow networks struggle with the nuance of ASL handshapes.

### *F. Trade-off*

The analysis reveals that amongst the models there is a trade-off between efficiency and computational cost. The CNN represented the fast training (5 mins), less computational requirements but lower accuracy (88%). This is the most ideal solution in real world with resource constraints. Transfer learning with the ResNet illustrated the opposite, slower training (1 hr), considerable requirements and near prefect accuracy (99.9%). The vision transformer sat in between with great accuracy (99.2%), moderate speed (20 mins) and computational cost. This highlights that computational overhead of attention mechanism is similar to simple convolutions

## V. DISCUSSION AND FUTURE WORK

### *A. Implications*

- Transformer model: We found that even with a compact version of the ViT transformer model, and a lower-than-average time in training, we could still achieve a near perfect score. This shows that even when not using a lot of computational resources, we can still achieve cutting edge results.

- The CNN model: The strong performance of the CNN model shows that for this kind of task; it is still competitive. With a potentially lighter offer than the other models, training from scratch, we can still achieve fantastic results.
- The CNN Transfer learning model: The Resnet 50 model, reached the highest accuracy of the three, but with the drawback of taking the longest time to train.

### *B. Analysis*

While we think that our different models gave a good variation of the types of results we could expect, if we were to train them with a wider variety ASL classes, we do note that some of them wouldn't be best suited to use if we wanted to maintain the same kind of accuracy. For instance the CNN model, while effective for this project, wouldn't be as good at capturing key data, as say the transformer model, with its ability in its self-attention mechanism to weight the importance of all parts of an image, which could theoretically learn many more delicate features of specific finger positions or wrist orientations.

### C. *FUTURE WORKS:*

We felt that these promising results would be a great starting point to develop our models further. Some areas we considered are;

- Training the model on a more diverse set of data, such as non-manual signals or co-articulation between signals.

- Training the models to output entire words and sentences from a series of ASL hand signals.

- Training the model of more realistic depictions (ASL in real word settings).

- Developing the models to work on long series of images, i.e., video recordings.

- Develop the models to understand different sign languages (BSL, LSF etc).

### D. LIMITATIONS

Some potential limitations we concluded would be the data set that the models were trained on. While it was a large data set, the images we're very isolated and static, featuring the same persons hand with only slight variations within each class, with rather clean backgrounds. This we feel wouldn't translate all that well into real world scenarios, where different skin tones, lighting and angles would make it rather hard for our models to cope.

We also on focus on hand signals, this removes all the other expressions, such as facial, mouth or even body movements, which leaves a gap in properly interpreting what a person may be trying to convey.

We also noticed that if we want to train our models from scratch, it took a very long time to train, the transformer, which is why we opted to use a base model instead of from scratch. If we wanted to further adapt it as discussed above, we would need to work out how we would be able to train effectively, given that to train some of the models, it takes a lot of computational resources.

## VI. Acknowledgment

Generative AI tools (such as ChatGPT, Grammarly) were used by all team members to support the development of this project. Their use was limited to tasks such as debugging code, refining explanations, and helping articulate lengthy sections more concisely.

## References

- B. Alsharif, A. S. Altaher, A. Altaher, M. Ilyas, and E. Alalwany. "Deep learning technology to recognize American Sign Language alphabet," Sensors, vol. 23, no. 18, pp. 7970, 2023.
- M. Al-Qurishi, T. Khalid, and R. Souissi. "Deep learning for sign language recognition: current techniques, benchmarks, and open issues," IEEE Access, vol. 9, p. 126917–126951, 2021.
- W. Tao, M. C. Leu, Z. Yin "American Sign Language alphabet recognition using Convolutional Neural Networks with multiview augmentation and inference fusion," Engineering Applications of Artificial Intelligence, vol. 79, p. 202-213, 2018.
- F. Chollet et al., "Keras," 2015. [Online]. Available: https://keras.io.
- A. Dosovitskiy et al. "An image is worth 16x16 words: Transformers for Image recognition at scale," in Int. Conf. Learn. Represent.,2021.
- A. Paszke et al., "Pytorch: An imperative Style, high-performance deep learning library," in Adv. Neural Inf. Process. Syst. 32, 2019, pp. 8024-8035.
- A. Nagaraj, "ASL alphabet," Kaggle , 2018. [Online]. Available: https://www.kaggle.com/datasets/grassknoted/asl-alphabet
- Huggingface.co 2025. WinKawaks/vit-tiny-patch16-224 [Online] Available: https://huggingface.co/WinKawaks/vit-tiny-patch16-224..